



Developing BookStore V-1.6

- In 1.6 Version of BookStore, we will add the following skills learned.
 - Cloud Config Server
 - Cloud Config Client
 - Cloud Bus
 - Profiles

A) Setup the Git Repository

1) Install the GIT

2) Create the folder structure required.

```
E:\mybookstore\myrepo>
```

3) Go that folder in the command line and run the following commands

```
git config --global user.email srinivas@coursecube.com
```

```
git config --global user.name "Srinivas Dande"
```

```
git init .
```

B) Setup the Cloud Config Server

4) Copy MyConfigServer from Cloud-Config-Demo to BookStore-V-1.6\MyLabs

5) Import MyConfigServer

C) Update BookPriceMS – V-1.6

6) Create the Property File called MyBookPriceMS.properties under GIT Repository

7) Copy following Properties into MyBookPriceMS.properties

```
management.endpoints.web.exposure.include=*  
management.endpoint.health.show-details=always
```



```
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

8) Create the Property File called MyBookPriceMS-dev.properties under GIT Repository for Development Environment

9) Copy following Properties into MyBookPriceMS-dev.properties

jlc.book.offer=50

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-book-price.log
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbookpricedb
spring.datasource.username=root
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

```
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

10) Create the Property File called MyBookPriceMS-prod.properties under GIT Repository for Production Environment

11) Copy following Properties into MyBookPriceMS-prod.properties

jlc.book.offer=25

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-book-price.log
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbookpricedb
spring.datasource.username=root
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

```
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

12) Run the following Git Commands

```
git status
git add .
git commit -m "Update #1 for MyBookPriceMS"
git status
```

13) Rename the application.properties of BookPriceMS to bootstrap.properties

14) bootstrap.properties should have the following properties

```
#src/main/resources/bootstrap.properties
```

```
spring.application.name=MyBookPriceMS
server.port=9000
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

15) Add the Cloud Config Client dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```



16) Add the Cloud Bus dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

17) Update BookPriceServiceImpl to use the offer from property file.

```
@Service
@Transactional
@RefreshScope //1
public class BookPriceServiceImpl implements BookPriceService {

    @Autowired
    BookPriceDAO bookPriceDAO;

    @Value("${jlc.book.offer}")
    String myoffer; //2

    static Logger log = LoggerFactory.getLogger(BookPriceServiceImpl.class);

    @Override
    @Transactional(readOnly = true)
    public BookPrice getBookPriceById(Integer bookId) {
        log.info("---BookPriceServiceImpl---getBookPriceById()-----");

        Optional<BookPrice> bookPriceOpts=bookPriceDAO.findById(bookId);
        BookPrice bookPrice = null;
        if(bookPriceOpts.isPresent()) {
            bookPrice = bookPriceOpts.get();
        }

        int myspecialOffer = Integer.parseInt(myoffer); //3
        System.out.println("myspecialOffer : -- "+myspecialOffer);
        System.out.println("bookPrice.getOffer() : -- "+bookPrice.getOffer());

        if( myspecialOffer > bookPrice.getOffer()) {
            bookPrice.setOffer(myspecialOffer);
        }
        return bookPrice;
    }
}
```

18) Remove the Hystrix Related Updates happned in BookStore-V-1.5.



BookPriceMS – V-1.6: Files required

1. BookPriceController.java	Updated in V1.6
2. BookPriceService.java	Same as V.1.5
3. BookPriceServiceImpl.java	Updated in V1.6
4. BookPriceDAO.java	Same as V.1.5
5. BookPrice.java	Same as V.1.5
6. JLCBookPriceConfig.java	Same as V.1.5
7. BookPriceApplication.java	Updated in V1.6
8. bootstrap.properties	Updated in V1.6
9. pom.xml	Updated in V1.6

1. BookPriceController.java

```
package com.jlcindia.bookprice;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RestController;

import io.swagger.annotations.ApiOperation;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@CrossOrigin
@RestController
public class BookPriceController {

    static Logger log = LoggerFactory.getLogger(BookPriceController.class);

    @Autowired
    BookPriceService bookPriceService;

    @GetMapping("/bookPrice/{bookId}")
    @ApiOperation(value = "getBookPrice", response = BookPrice.class, notes = "Gives BookPrice Object of given bookId")
```



```
public BookPrice getBookPrice(@PathVariable Integer bookId) {
log.info("---BookPriceController---getBookPrice()-----");

BookPrice bookPrice = bookPriceService.getBookPriceById(bookId);

return bookPrice;
}

@GetMapping("/offeredPrice/{bookId}")
@ApiOperation(value = " getOfferedPrice", response = double.class, notes = "Gives Offered
Price of given bookId")
public double getOfferedPrice(@PathVariable Integer bookId) {
log.info("---BookPriceController---getOfferedPrice()-----");

double offeredPrice = bookPriceService.getOfferedPriceById(bookId);

return offeredPrice;
}
}
```

3. BookPriceServiceImpl.java

```
package com.jlcindia.bookprice;

import java.util.Optional;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.cloud.context.config.annotation.RefreshScope;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@Service
@Transactional
@RefreshScope
public class BookPriceServiceImpl implements BookPriceService {
```



```
@Autowired
BookPriceDAO bookPriceDAO;

@Value("${jlc.book.offer}")
String myoffer;

static Logger log = LoggerFactory.getLogger(BookPriceServiceImpl.class);

@Override
@Transactional(readOnly = true)
public BookPrice getBookPriceById(Integer bookId) {
    log.info("---BookPriceServiceImpl---getBookPriceById()-----");

    Optional<BookPrice> bookPriceOpts=bookPriceDAO.findById(bookId);
    BookPrice bookPrice = null;
    if(bookPriceOpts.isPresent()) {
        bookPrice = bookPriceOpts.get();
    }

    int myspecialOffer = Integer.parseInt(myoffer);
    System.out.println("myspecialOffer : -- "+myspecialOffer);
    System.out.println("bookPrice.getOffer() : -- "+bookPrice.getOffer());

    if( myspecialOffer > bookPrice.getOffer()) {
        bookPrice.setOffer(myspecialOffer);
    }

    System.out.println(bookPrice);
    return bookPrice;
}

@Override
@Transactional(readOnly = true)
public double getOfferedPriceById(Integer bookId) {
    log.info("---BookPriceServiceImpl---getOfferedPriceById()-----");

    int myspecialOffer = Integer.parseInt(myoffer);
    System.out.println("myspecialOffer : -- "+myspecialOffer);

    Optional<BookPrice> bookPriceOpts=bookPriceDAO.findById(bookId);
    double offeredPrice = 0.0;

    if(bookPriceOpts.isPresent()) {
        BookPrice bookPrice = bookPriceOpts.get();
```



```
double actualPrice = bookPrice.getPrice();
double offer= bookPrice.getOffer();
double discountAmount =0.0;

if(myspecialOffer> offer) {
    discountAmount= actualPrice * myspecialOffer /100;
}else {
    discountAmount = actualPrice * offer /100;
}

offeredPrice = actualPrice - discountAmount;
}

return offeredPrice;
}

}
```

7. BookPriceApplication.java

```
package com.jlcindia.bookprice;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;

import springfox.documentation.swagger2.annotations.EnableSwagger2;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */

@SpringBootApplication
@EnableSwagger2
@EnableEurekaClient
public class BookPriceApplication {
    public static void main(String args[]) {
        SpringApplication.run(BookPriceApplication.class, args);
    }
}
```



8. bootstrap.properties

```
spring.application.name=MyBookPriceMS
server.port=9000
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

9. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookprice</groupId>
<artifactId>BookPriceMS</artifactId>
<version>1.6</version>
<packaging>jar</packaging>

<name>MyBookPriceMS</name>
<url>https://www.jlcindia.com</url>
<description>This is BookPrice MS developed by Srinivas Dande
</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>
```



```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```

D) Update BookSearchMS – V-1.6

19) Create the Property File called MyBookSearchMS.properties under GIT Repository

20) Copy following Properties into MyBookSearchMS.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

21) Create the Property File called MyBookSearchMS-dev.properties under GIT Repository for Development Environment

22) Copy following Properties into MyBookSearchMS-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.boot.admin.client.url=http://localhost:5555

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-book-search.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

23) Create the Property File called MyBookSearchMS-prod.properties under GIT Repository for Production Environment

24) Copy following Properties into MyBookSearchMS-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka  
eureka.client.healthcheck.enabled=true
```

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000  
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO  
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-book-search.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

25) Run the following Git Commands

```
git status  
git add .  
git commit -m "Update #2 for MyBookSeacrMS"  
git status
```



26) Rename the application.properties of BookSearchMS to bootstrap.properties

27) bootstrap.properties should have the following properties

```
#src/main/resources/bootstrap.properties

spring.application.name=MyBookSearchMS
server.port=8000
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

28) Add the Cloud Config Client dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

29) Add the Cloud Bus dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

BookSearchMS – V-1.6: Files required

1. BookController.java	Same as V1.5
2. BookPriceInfo.java	Same as V1.5
3. BookInfo.java	Same as V1.5
4. BookInventoryInfo.java	Same as V1.5
5. BookRatingInfo.java	Same as V1.5
6. BookPriceProxy.java	Same as V1.5
7. BookService.java	Same as V1.5
8. BookServiceImpl.java	Same as V1.5
9. BookDAO.java	Same as V1.5
10. BookRatingDAO.java	Same as V1.5
11. BookInventoryDAO.java	Same as V1.5
12. Book.java	Same as V1.5
13. BookRating.java	Same as V1.5
14. BookInventory.java	Same as V1.5



15. JLCBookSearchConfig.java	Same as V1.5
16. BookSearchApplication.java	Same as V1.5
17.bootstrap.properties	Updated in V1.6
18.pom.xml	Updated in V1.6

17. bootstrap.properties

```
spring.application.name=MyBookSearchMS  
server.port=8000  
spring.cloud.config.uri=http://localhost:2345  
spring.profiles.active=dev
```

18. pom.xml

```
<project ...>  
<modelVersion>4.0.0</modelVersion>  
<groupId>com.jlcindia.booksearch</groupId>  
<artifactId>BookSearchMS</artifactId>  
<version>1.6</version>  
<packaging>jar</packaging>  
  
<name>MyBookSearchMS</name>  
<url>https://www.jlcindia.com</url>  
<description>This is BookSeach MS developed by Srinivas Dande  
</description>  
  
<properties>  
<java.version>1.8</java.version>  
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>  
</properties>  
  
<parent>  
<groupId>org.springframework.boot</groupId>  
<artifactId>spring-boot-starter-parent</artifactId>  
<version>2.3.3.RELEASE</version>  
</parent>  
  
<dependencies>  
  
<dependency>  
<groupId>org.springframework.cloud</groupId>  
<artifactId>spring-cloud-starter-bus-amqp</artifactId>  
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```



```
<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



E)Update PlaceOrderMS – V-1.6

30) Create the Property File called MyPlaceOrderMS.properties under GIT Repository

31) Copy following Properties into MyPlaceOrderMS.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

32) Create the Property File called MyPlaceOrderMS-dev.properties under GIT Repository for Development Environment

33) Copy following Properties into MyPlaceOrderMS-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-place-order.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

34) Create the Property File called MyPlaceOrderMS-prod.properties under GIT Repository for Production Environment

35) Copy following Properties into MyPlaceOrderMS-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka  
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000  
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO  
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-place-order.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

36) Run the following Git Commands

```
git status  
git add .  
git commit -m "Update #3 for PlaceOrderMS"  
git status
```

37) Rename the application.properties of PlaceOrderMS to bootstrap.properties



38) bootstrap.properties should have the following properties

#src/main/resources/bootstrap.properties

```
spring.application.name=MyPlaceOrderMS
server.port=7000
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

39) Add the Cloud Config Client dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

40) Add the Cloud Bus dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

PlaceOrderMS – V-1.6: Files required

1. OrderController.java	Same as V1.5
2. OrderFullInfo.java	Same as V1.5
3. OrderInfo.java	Same as V1.5
4. OrderItemInfo.java	Same as V1.5
5. BookInventoryInfo.java	Same as V1.5
6. OrderService.java	Same as V1.5
7. OrderServiceImpl.java	Same as V1.5
8. OrderDAO.java	Same as V1.5
9. OrderItemDAO.java	Same as V1.5
10. BookInventoryDAO.java	Same as V1.5
11. Order.java	Same as V1.5
12. OrderItem.java	Same as V1.5
13. BookInventory.java	Same as V1.5
14. JLCPlaceOrderConfig.java	Same as V1.5
15. PlaceOrderApplication.java	Same as V1.5
16. bootstrap.properties	Updated in V1.6
17. pom.xml	Updated in V1.6



16. bootstrap.properties

```
spring.application.name=MyPlaceOrderMS
server.port=7000
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

17. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.placeorder</groupId>
<artifactId>PlaceOrderMS</artifactId>
<version>1.6</version>
<packaging>jar</packaging>

<name>MyPlaceOrderMS</name>
<url>https://www.jlcindia.com</url>
<description>This is PlaceOrderMS developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>
```



```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



F)Update UserRatingMS – V-1.6

41) Create the Property File called MyUserRatingMS.properties under GIT Repository

42) Copy following Properties into MyUserRatingMS.properties

```
management.endpoints.web.exposure.include=*  
management.endpoint.health.show-details=always
```

```
info.app.name=@project.name@  
info.app.description=@project.description@  
info.app.version=@project.version@  
info.app.encoding=@project.build.sourceEncoding@  
info.app.java.version=@java.version@  
info.app.developer=Srinivas Dande  
info.app.company=Java Learning Center
```

43) Create the Property File called MyUserRatingMS-dev.properties under GIT Repository for Development Environment

44) Copy following Properties into MyUserRatingMS-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka  
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000  
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO  
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-user-rating.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

45) Create the Property File called MyUserRatingMS-prod.properties under GIT Repository for Production Environment

46) Copy following Properties into MyUserRatingMS-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka  
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000  
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO  
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-user-rating.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

47) Run the following Git Commands

```
git status  
git add .  
git commit -m "Update #4 for UserRatingMS"  
git status
```

48) Rename the application.properties of UserRatingMS to bootstrap.properties

49) bootstrap.properties should have the following properties

#src/main/resources/bootstrap.properties

```
spring.application.name=MyUserRatingMS
server.port=6500
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

50) Add the Cloud Config Client dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

51) Add the Cloud Bus dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

UserRatingMS – V-1.6: Files required

1. RatingController.java	Same as V1.5
2. UserRatingInfo.java	Same as V1.5
3. BookRatingInfo.java	Same as V1.5
4. RatingService.java	Same as V1.5
5. RatingServiceImpl.java	Same as V1.5
6. UserRatingDAO.java	Same as V1.5
7. BookRatingDAO.java	Same as V1.5
8. UserRating.java	Same as V1.5
9. BookRating.java	Same as V1.5
10. JLCUserRatingConfig.java	Same as V1.5
11. UserRatingApplication.java	Same as V1.5
12.bootstrap.properties	Updated in V1.6
13.pom.xml	Updated in V1.6



12. bootstrap.properties

```
spring.application.name=MyUserRatingMS
server.port=6500
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

13. pom.xml

```
<project ...>

<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.userrating</groupId>
<artifactId>UserRatingMS</artifactId>
<version>1.6</version>
<packaging>jar</packaging>

<name>MyUserRatingMS</name>
<url>https://www.jlcindia.com</url>
<description>This is UserRatingMS developed by Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>

<dependency>
<groupId>mysql</groupId>
<artifactId>mysql-connector-java</artifactId>
<version>5.1.6</version>
</dependency>

<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger2</artifactId>
<version>2.4.0</version>
</dependency>
```



```
<dependency>
<groupId>io.springfox</groupId>
<artifactId>springfox-swagger-ui</artifactId>
<version>2.4.0</version>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>

</project>
```



G)Update BookStoreWeb – V-1.6

52) Create the Property File called MyBookStoreWeb.properties under GIT Repository

53) Copy following Properties into MyBookStoreWeb.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

54) Create the Property File called MyBookStoreWeb-dev.properties under GIT Repository for Development Environment

55) Copy following Properties into MyBookStoreWeb-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```



```
logging.file.name=E:/mylogs/jlc-user-rating.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

56) Create the Property File called MyBookStoreWeb-prod.properties under GIT Repository for Production Environment

57) Copy following Properties into MyBookStoreWeb-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka  
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000  
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO  
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-user-rating.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```

58) Run the following Git Commands

```
git status  
git add .  
git commit -m "Update #4 for UserRatingMS"  
git status
```

59) Rename the application.properties of BookStoreWeb to bootstrap.properties

60) bootstrap.properties should have the following properties

#src/main/resources/bootstrap.properties

```
spring.application.name=MyUserRatingMS
server.port=6500
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

61) Add the Cloud Config Client dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```

62) Add the Cloud Bus dependency to the pom.xml file

```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>
```

BookStoreWeb – V-1.6: Files required

1. index.jsp	Same as V1.5
2. myheader.jsp	Same as V1.5
3. myfooter.jsp	Same as V1.5
4. showBooksList.jsp	Same as V1.5
5. showBookInfo.jsp	Same as V1.5
6. showCart.jsp	Same as V1.5
7. orderSuccess.jsp	Same as V1.5
8. addRating.jsp	Same as V1.5
9. ratingSuccess.jsp	Same as V1.5
10. ratingsList.jsp	Same as V1.5
11. bookstore.css	Same as V1.5
12. Book.java	Same as V1.5
13. BookInfo.java	Same as V1.5
14. Order.java	Same as V1.5



15. OrderItem.java	Same as V1.5
16. UserRating.java	Same as V1.5
17.BookStoreController.java	Updated in V1.6
18. OrderFullInfo.java	Same as V1.5
19. OrderInfo.java	Same as V1.5
20. OrderItemInfo.java	Same as V1.5
21. UserRatingInfo.java	Same as V1.5
22. BookPriceInfo.java	Same as V1.5
23. BookSearchProxy.java	Same as V1.5
24. PlaceOrderProxy.java	Same as V1.5
25. UserRatingProxy.java	Same as V1.5
26.BookStoreService.java	Updated in V1.6
27.BookStoreServiceImpl.java	Updated in V1.6
28. JLCBookStoreWebConfig.java	Same as V1.5
29. BookStoreWebApplication.java	Same as V1.5
30.bootstrap.properties	Updated in V1.6
31.pom.xml	Updated in V1.6

26. BookStoreService.java

```
package com.jlcindia.bookstoreweb;
```

```
import java.util.*;
```

```
/*
```

```
* @Author : Srinivas Dande
```

```
* @company : Java Learning Center
```

```
**/
```

```
public interface BookStoreService {
```

```
    public List<String> getAuthorsList();
```

```
    public List<String> getCategoryList();
```

```
    public List<Book> getMyBooks(String author, String category);
```

```
    public BookInfo getBookInfoByBookId(Integer bookId);
```

```
    public void placeOrder( Map<Integer,BookInfo> mycartMap);
```

```
    public List<Order> getMyOrders(String userId);
```

```
    public void addUserRating(UserRating userRating);
```

```
    public List<UserRating> getMyRatings(String userId);
```

```
}
```



27. BookStoreServiceImpl.java

```
package com.jlcindia.bookstoreweb;

import java.text.SimpleDateFormat;
import java.util.*;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.amqp.rabbit.core.RabbitTemplate;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.jlcindia.rabbitmq.OrderFullInfo;
import com.jlcindia.rabbitmq.OrderInfo;
import com.jlcindia.rabbitmq.OrderItemInfo;
import com.jlcindia.rabbitmq.UserRatingInfo;

@Service
public class BookStoreServiceImpl implements BookStoreService {

    static Logger log = LoggerFactory.getLogger(BookStoreServiceImpl.class);

    @Autowired
    RabbitTemplate rabbitTemplate;

    @Autowired
    BookSearchProxy bookSearchProxy;

    @Autowired
    PlaceOrderProxy placeOrderProxy;

    @Autowired
    UserRatingProxy userRatingProxy;

    public List<String> getAuthorsList() {
        List<String> authorsList = new ArrayList<>();
        authorsList.add("All Authors");
        authorsList.add("Srinivas");
        authorsList.add("Vas");
        authorsList.add("Sri");
        return authorsList;
    }
}
```



```
public List<String> getCategoryList() {
    List<String> catList = new ArrayList<>();
    catList.add("All Categories");
    catList.add("Web");
    catList.add("Spring");
    return catList;
}

public List<Book> getMyBooks(String author, String category) {
    System.out.println("BookStoreServiceImpl - getBooks()");

    if (author == null || author.length() == 0) {
        author = "All Authors";
    }
    if (category == null || category.length() == 0) {
        category = "All Categories";
    }

    //Invoking BookSearch Rest API with Feign
    List<Book> bookList = bookSearchProxy.getBooks(author, category);
    return bookList;
}

public BookInfo getBookInfoByBookId(Integer bookId) {
    System.out.println("BookStoreServiceImpl - getBookInfoByBookId()");

    //Invoking BookSearch Rest API with Feign
    BookInfo bookInfo = bookSearchProxy.getBookById(bookId);
    return bookInfo;
}

public void placeOrder(Map<Integer, BookInfo> mycartMap) {

    System.out.println("---2.BookStoreServiceImpl--placeOrder()----");
    List<OrderItemInfo> itemList = new ArrayList<>();
    double totalPrice = 0.0;
    int totalQuantity = 0;

    for (BookInfo mybook : mycartMap.values()) {
        Integer bookId = mybook.getBookId();

        // Invoke BookPriceMS Rest API with Feign
        double offerPrice = bookSearchProxy.getOfferedPrice(bookId);
```



```
OrderItemInfo item = new OrderItemInfo(0, bookId, 1, offerPrice);
itemList.add(item);

totalPrice = totalPrice + offerPrice;
totalQuantity = totalQuantity + 1;
}

Date today = Calendar.getInstance().getTime();
SimpleDateFormat formatter = new SimpleDateFormat("dd-MM-yyyy hh:mm");
String orderDate = formatter.format(today);
System.out.println(orderDate);

OrderInfo orderInfo = new OrderInfo(orderDate, "U-123", totalQuantity, totalPrice, "New");

OrderFullInfo orderFullInfo = new OrderFullInfo();
orderFullInfo.setOrder(orderInfo);
orderFullInfo.setItemList(itemList);

// Sending Order Message to RabbitMQ
rabbitTemplate.convertAndSend("myorder.exchange","myorder.key", orderFullInfo);
System.out.println("Order Placed");
}

@Override
public List<Order> getMyOrders(String userId) {

// Invoke PlaceOrderMS Rest API with Feign
List<Order> myorders = placeOrderProxy.getOrdersByUserId(userId);

return myorders;
}

public void addUserRating(UserRating userRating) {
System.out.println("---2.BookStoreServiceImpl--addUserRating()----");
//Sending UserRating Message RabbitMQ
UserRatingInfo userRatingInfo=new UserRatingInfo(userRating.getUserId(),
userRating.getBookId(), userRating.getRating(), userRating.getReview());
rabbitTemplate.convertAndSend("myuser.ratings.exchange","myuser.ratings.key",
userRatingInfo);

System.out.println("Rating Added");
}
```



```
public List<UserRating> getMyRatings(String userId) {  
  
    // Invoke UserRatingMS Rest API with Feign  
    List<UserRating> ratingsList = userRatingProxy.getUserRatingByUserId(userId);  
  
    return ratingsList;  
}  
}
```

17. BookStoreService.java

```
package com.jlcindia.bookstoreweb;  
  
Importas as here  
/*  
 * @Author : Srinivas Dande  
 * @company : Java Learning Center  
 */  
@Controller  
public class BookStoreController {  
  
    //Remaining Methods Same  
  
    @PostMapping("/addToCart")  
    public String addBookToCart(@RequestParam("bookId") String bookId, HttpSession session)  
    {  
        System.out.println("-----BookStoreController--addBookToCart()-----");  
        System.out.println(bookId);  
  
        //Book mybook = bookStoreService.getBookByBookId(new Integer(bookId));  
  
        BookInfo mybookInfo = bookStoreService.getBookInfoByBookId(new  
        Integer(bookId));  
        System.out.println(mybookInfo);  
        mycartMap.put(new Integer(bookId), mybookInfo);  
        session.setAttribute("MyCart", mycartMap);  
        return "showBooksList";  
    }  
  
    //Remaining Methods Same  
}
```



30. bootstrap.properties

```
spring.application.name=MyBookStoreWeb
server.port=5000
spring.cloud.config.uri=http://localhost:2345
spring.profiles.active=dev
```

31.pom.xml

```
<project .....">
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.bookstoreweb</groupId>
<artifactId>BookStoreWeb</artifactId>
<version>1.6</version>
<packaging>jar</packaging>

<name>MyBookStoreWeb</name>
<url>https://www.jlcindia.com</url>
<description>This is BookStoreWeb developed by Srinivas Dande
</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-bus-amqp</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-config</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-amqp</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-client</artifactId>
<version>2.3.0</version>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>org.apache.tomcat.embed</groupId>
<artifactId>tomcat-embed-jasper</artifactId>
</dependency>

<dependency>
<groupId>jstl</groupId>
<artifactId>jstl</artifactId>
```



```
<version>1.2</version>
</dependency>

<dependency>
<groupId>taglibs</groupId>
<artifactId>standard</artifactId>
<version>1.1.2</version>
</dependency>

<dependency>
<groupId>org.webjars</groupId>
<artifactId>bootstrap</artifactId>
<version>4.3.1</version>
</dependency>

<dependency>
<groupId>org.webjars</groupId>
<artifactId>jquery</artifactId>
<version>3.4.1</version>
</dependency>
</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



H) MyEurekaServer – V-1.6

MyEurekaServer – Files Required

1. MyEurekaServerApplication.java	2. application.properties
3. pom.xml	

1. MyEurekaServerApplication.java

```
package com.jlcindia.myeurekaserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.server.EnableEurekaServer;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaServer
public class MyEurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyEurekaServerApplication.class, args);
    }
}
```

2. application.properties

```
spring.application.name=MyEurekaServer
server.port=8761

eureka.client.register-with-eureka=false
eureka.client.fetchRegistry= false
```

3. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.myeurekaserver</groupId>
<artifactId>MyEurekaServer</artifactId>
<version>1.6</version>
<name>MyEurekaServer</name>
<description>Eureka - Service Registry and Discovery Server</description>

<properties>
<java.version>1.8</java.version>
```



```
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



I) MyConfigServer – V-1.6

MyConfigServer – Files Required

1. MyConfigServerApplication.java	2. bootstrap.properties
3. pom.xml	

1. MyConfigServerApplication.java

```
package com.jlcindia.myconfigserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.config.server.EnableConfigServer;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableConfigServer
public class MyConfigServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyConfigServerApplication.class, args);
    }
}
```

2. bootstrap.properties

```
spring.application.name=MyConfigServer
server.port=2345
spring.cloud.config.server.git.uri = E:/mybookstore/myrepo
```

3. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.myconfigserver</groupId>
<artifactId>MyConfigServer</artifactId>
<version>1.6</version>
<name>MyConfigServer</name>
<description>ConfigServer - By Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
```



```
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-config-server</artifactId>
</dependency>

</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



J) MyApiGatewayServer- V-1.6

MyApiGatewayServer – Files Required

1. MyGatewayServerApplication.java	2. MyLogFilter.java
3. MySecurityFilter.java	4. application.properties
5. pom.xml	

1. MyGatewayServerApplication.java

```
package com.jlcindia.mygatewayserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.cloud.netflix.eureka.EnableEurekaClient;
import org.springframework.cloud.netflix.zuul.EnableZuulProxy;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
@SpringBootApplication
@EnableEurekaClient
@EnableZuulProxy
public class MyGatewayServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyGatewayServerApplication.class, args);
    }
}
```

2. MyLogFilter.java

```
package com.jlcindia.mygatewayserver;

import javax.servlet.http.HttpServletRequest;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Component;

import com.netflix.zuul.ZuulFilter;
import com.netflix.zuul.context.RequestContext;
import com.netflix.zuul.exception.ZuulException;
/*
 * @Author : Srinivas Dande
 * @company : Java Learning Center
 */
```



```
*/
```

```
@Component
```

```
public class MyLogFilter extends ZuulFilter{
```

```
    Logger logger=LoggerFactory.getLogger(this.getClass());
```

```
    @Override
```

```
    public Object run() throws ZuulException {
```

```
        logger.info("---1. MyLogFilter---run()");
```

```
        // Write your logic Here
```

```
        HttpServletRequest request=RequestContext.getCurrentContext().getRequest();
```

```
        String URI=request.getRequestURI();
```

```
        logger.info(" Request From => "+URI);
```

```
        return null;
```

```
    }
```

```
    @Override
```

```
    public String filterType() {
```

```
        return "pre";
```

```
    }
```

```
    @Override
```

```
    public int filterOrder() {
```

```
        return 1;
```

```
    }
```

```
    @Override
```

```
    public boolean shouldFilter() {
```

```
        return true;
```

```
    }
```

```
    }
```

3. MySecurityFilter.java

```
package com.jlcindia.mygatewayserver;
```

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
import org.springframework.stereotype.Component;
```

```
import com.netflix.zuul.ZuulFilter;
```

```
import com.netflix.zuul.exception.ZuulException;
```

```
/*
```

```
 * @Author : Srinivas Dande
```



```
* @company : Java Learning Center
```

```
**/
```

```
@Component
```

```
public class MySecurityFilter extends ZuulFilter{
```

```
    Logger logger=LoggerFactory.getLogger(this.getClass());
```

```
@Override
```

```
public Object run() throws ZuulException {
```

```
    logger.info("---2. MySecurityFilter---run()");
```

```
    // Write your logic Here
```

```
    logger.info("Authetication Done Successfully");
```

```
    return null;
```

```
}
```

```
@Override
```

```
public String filterType() {
```

```
    return "pre";
```

```
}
```

```
@Override
```

```
public int filterOrder() {
```

```
    return 2;
```

```
}
```

```
@Override
```

```
public boolean shouldFilter() {
```

```
    return true;
```

```
}
```

```
}
```

4. application.properties

```
spring.application.name=MyApiGatewayServer
```

```
server.port=5678
```

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
```

```
eureka.client.healthcheck.enabled=true
```

```
zuul.routes.book-price.path=/myjlc-book-price/**
```

```
zuul.routes.book-price.url=MyBookPriceMS
```



```
zuul.routes.book-search.path=/myjlc-book-search/**
zuul.routes.book-search.url=MyBookSearchMS

zuul.routes.book-store.path=/myjlc-book-store/**
zuul.routes.book-store.url=MyBookStoreWeb

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-gate-way.log

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

5. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.mygatewayserver</groupId>
<artifactId>MyApiGatewayServer</artifactId>
<version>1.0</version>
<name>MyApiGatewayServer</name>
<description> Zuul - MyApiGatewayServer</description>

<properties>
<java.version>1.8</java.version>
<spring-cloud.version>Hoxton.SR8</spring-cloud.version>
</properties>

<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```



```
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-netflix-zuul</artifactId>
</dependency>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
</dependencies>

<dependencyManagement>
<dependencies>
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-dependencies</artifactId>
<version>${spring-cloud.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



K) MyBootAdminServer – V-1.6

MyBootAdminServer – Files Required

1. MyBootAdminServerApplication.java	2. application.properties
3. pom.xml	

1. MyBootAdminServerApplication.java

```
package com.jlcindia.mybootadminserver;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import de.codecentric.boot.admin.server.config.EnableAdminServer;

@SpringBootApplication
@EnableAdminServer
public class MyBootAdminServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(MyBootAdminServerApplication.class, args);
    }
}
```

2. application.properties

```
spring.application.name=MyBootAdminServer

server.port=5555
```

3. pom.xml

```
<project ...>
<modelVersion>4.0.0</modelVersion>
<groupId>com.jlcindia.mybootadminserver</groupId>
<artifactId>MyBootAdminServer</artifactId>
<version>1.6</version>
<name>MyBootAdminServer</name>
<description>This is Boot Admin Server Written By Srinivas Dande</description>

<properties>
<java.version>1.8</java.version>
<spring-boot-admin.version>2.3.0</spring-boot-admin.version>
</properties>
```



```
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.3.3.RELEASE</version>
</parent>

<dependencies>

<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-actuator</artifactId>
</dependency>

<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-starter-server</artifactId>
</dependency>

</dependencies>
<dependencyManagement>
<dependencies>
<dependency>
<groupId>de.codecentric</groupId>
<artifactId>spring-boot-admin-dependencies</artifactId>
<version>${spring-boot-admin.version}</version>
<type>pom</type>
<scope>import</scope>
</dependency>
</dependencies>
</dependencyManagement>

<build>
<plugins>
<plugin>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-maven-plugin</artifactId>
</plugin>
</plugins>
</build>
</project>
```



L) LogStach Configuration

Prepare the logstash configuration file as follows

1) jlc-logstash.conf

```
input {  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-book-price.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-book-search.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-book-store.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"  
      what => "previous"  
    }  
  }  
  file {  
    type => "java"  
    path => "E:/mylogs/jlc-place-order.log"  
    codec => multiline {  
      pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"  
      negate => "true"
```



```
    what => "previous"
  }
}
file {
  type => "java"
  path => "E:/mylogs/jlc-user-rating.log"
  codec => multiline {
    pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"
    negate => "true"
    what => "previous"
  }
}
file {
  type => "java"
  path => "E:/mylogs/jlc-gate-way.log"
  codec => multiline {
    pattern => "^%{YEAR}-%{MONTHNUM}-%{MONTHDAY} %{TIME}.*"
    negate => "true"
    what => "previous"
  }
}
}

filter {
  if [message] =~ "\tat" {
    grok {
      match => ["message", "^(\\tat)"]
      add_tag => ["my-jlc-error-trace"]
    }
  }
}

output {
  stdout {
    codec => rubydebug
  }
  elasticsearch {
    hosts => ["localhost:9200"]
  }
}
```



M) All the Files in Git Repo

Git Location : E:/mybookstore/myrepo

1. MyBookPriceMS.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always
```

```
# Info Endpoint Config
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

2. MyBookPriceMS-dev.properties

jlc.book.offer=50

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n
```

logging.file.name=E:/mylogs/jlc-book-price.log

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbookpricedb
spring.datasource.username=root
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5
```

```
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```



3. MyBookPriceMS-prod.properties

jlc.book.offer=25

eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-book-price.log

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbookpricedb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update

4. MyBookSearchMS.properties

management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

Info Endpoint Config
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center



5. MyBookSearchMS-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.boot.admin.client.url=http://localhost:5555

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-book-search.log

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

6. MyBookSearchMS-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.boot.admin.client.url=http://localhost:5555

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcbooksdb
spring.datasource.username=root
spring.datasource.password=srinivas
```



```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-book-search.log

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

7. MyPlaceOrderMS.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

# Info Endpoint Config
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```

8. MyPlaceOrderMS-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb
spring.datasource.username=root
spring.datasource.password=srinivas
```



```
spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-place-order.log

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

9. MyPlaceOrderMS-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcordersdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-place-order.log

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```



10. MyUserRatingMS.properties

```
management.endpoints.web.exposure.include=*  
management.endpoint.health.show-details=always
```

```
# Info Endpoint Config  
info.app.name=@project.name@  
info.app.description=@project.description@  
info.app.version=@project.version@  
info.app.encoding=@project.build.sourceEncoding@  
info.app.java.version=@java.version@  
info.app.developer=Srinivas Dande  
info.app.company=Java Learning Center
```

11. MyUserRatingMS-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka  
eureka.client.healthcheck.enabled=true
```

```
spring.boot.admin.client.url=http://localhost:5555
```

```
spring.rabbitmq.host=localhost  
spring.rabbitmq.port=5672  
spring.rabbitmq.username=guest  
spring.rabbitmq.password=guest
```

```
spring.datasource.driver-class-name=com.mysql.jdbc.Driver  
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb  
spring.datasource.username=root  
spring.datasource.password=srinivas
```

```
spring.datasource.hikari.connectionTimeout=20000  
spring.datasource.hikari.maximumPoolSize=5
```

```
logging.level.root=INFO  
logging.pattern.console=%-5level %logger{36} - %msg%n
```

```
logging.file.name=E:/mylogs/jlc-user-rating.log
```

```
spring.jpa.show-sql=true  
spring.jpa.hibernate.ddl-auto=update
```



12. MyUserRatingMS-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

spring.datasource.driver-class-name=com.mysql.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/jlcratingsdb
spring.datasource.username=root
spring.datasource.password=srinivas

spring.datasource.hikari.connectionTimeout=20000
spring.datasource.hikari.maximumPoolSize=5

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-user-rating.log

spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
```

13. MyBookStoreWeb.properties

```
management.endpoints.web.exposure.include=*
management.endpoint.health.show-details=always

# Info Endpoint Config
info.app.name=@project.name@
info.app.description=@project.description@
info.app.version=@project.version@
info.app.encoding=@project.build.sourceEncoding@
info.app.java.version=@java.version@
info.app.developer=Srinivas Dande
info.app.company=Java Learning Center
```



14. MyBookStoreWeb-dev.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-book-store.log
```

15. MyBookStoreWeb-prod.properties

```
eureka.client.service-url.defaultZone=http://localhost:8761/eureka
eureka.client.healthcheck.enabled=true

spring.boot.admin.client.url=http://localhost:5555

spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest

logging.level.root=INFO
logging.pattern.console=%-5level %logger{36} - %msg%n

logging.file.name=E:/mylogs/jlc-book-store.log
```

N) Steps to Run BookStore-1.6

1) Start MySQL Server If not Started.

2) Start the RabbitMQ Server If not Started.

E:\MyRabbitMQ\rabbitmq_server-3.9.5\sbin>rabbitmq-server.bat

3) Start the ElasticSearch

E:\elasticsearch-7.15.2\bin>elasticsearch.bat

4) Start the Kibana

E:\kibana-7.15.2\bin>kibana.bat

5) Start the logStash

E:\logstash-7.15.2\bin>logstash -f jlc-logstash.conf

6) Start BookAdmin Server

7) Start Eureka Server

8) Start Gateway Server

9) Start Config Server

10) Start BookPriceMS

11) Start BookSearchMS

12) Start PlaceOrderMS

13) Start UserRatingMS

14) Start BookStoreWeb

<http://localhost:5000/>

<http://localhost:5678/myjlc-book-store/>